

CONVERA UI Manual

Overview

The CONVERA UI is a local FastAPI dashboard for running prompts and viewing high-level runtime metrics. It is designed for local testing, demos, and repeated-prompt validation.

The UI files are:

```
ui/app.py
ui/static/index.html
ui/dashboard.py
```

Start The UI

From the project root:

```
uvicorn ui.app:app --reload
```

Open the dashboard:

```
http://127.0.0.1:8000/static/index.html
```

Before You Use The UI

Make sure a model exists at:

```
models/llama3
```

If not, download one:

```
python scripts/download_llama.py
```

Some Hugging Face models require authentication and license acceptance.

UI Layout

The page has two main areas:

1. Prompt panel.
2. Metrics panel.

It also includes a benchmark graph panel below the main grid when a graph has been generated by `python -m metrics.benchmark_runner`.

Below the benchmark panel, the UI includes:

- Execution Inspector.
- Audit Trace.

The top header includes:

- CONVERA title.
- Tagline.
- Refresh button.

Prompt Panel

The prompt panel includes:

- A text area.
- A precision dropdown.
- A max tokens input.
- An audit trace checkbox.
- A Run button.
- An output display.

Default prompt:

Explain how neural networks work in detail.

How to run:

1. Edit the text area.
2. Choose precision: auto, fp16, int8, or int4.
3. Choose a max token count. For short demos, use 8 to 16.
4. Optionally enable Audit trace.
5. Click Run.
6. Wait for the model to generate.
7. Read the response in the output display.

While running, the button is disabled and the output box says:

Running...

If the request fails, the output box displays the error string.

Metrics Panel

The metrics panel shows the latest high-level runtime values:

- Latency.
- Tokens/sec.
- KV hit rate.
- Compute avoided.

Below the cards is a recent run table with:

- Run label.
- Latency.
- TPS.
- KV status.
- Precision.
- Compute avoided.

Benchmark Graph Panel

The benchmark graph panel displays:

`metrics/output/benchmark.png`

Generate it with:

```
python -m metrics.benchmark_runner
```

Then refresh the UI graph. The server exposes the graph at:

GET `/benchmark-graph`

If no graph exists yet, the UI shows a neutral empty state.

Execution Inspector Panel

The execution inspector shows the latest high-level request path:

- Mode.
- Precision.
- Validation status.
- Routing step.
- Precision step.
- Execution step.
- Validation step.

It uses public execution records only. It does not show prompts, outputs, raw tokens, local paths, private formulas, or private runtime internals.

Audit Trace Panel

The audit trace panel displays an exportable redacted trace for the selected request. Enable Audit trace before clicking Run to create a trace for that request.

The trace includes:

- Request ID.

- Status.
- Public execution steps.
- Redacted step data.

Use Download JSON to export the trace.

Refresh Button

Click Refresh to reload metrics manually.

The page also refreshes metrics automatically every 3 seconds.

API Endpoints Used By The UI

Health

GET /health

Example:

```
curl http://127.0.0.1:8000/health
```

Response shape:

```
{
  "ok": true,
  "engine_loaded": false
}
```

engine_loaded becomes true after the runtime has loaded the model.

Status

GET /status

Example:

```
curl http://127.0.0.1:8000/status
```

Response shape:

```
{
  "ok": true,
  "engine_loaded": false,
  "verification_mode": false,
}
```

```
  "audit_mode": false
}
```

Metrics

GET /metrics

Example:

```
curl http://127.0.0.1:8000/metrics
```

Returns recent dashboard history. The UI displays the newest row in the metric cards and recent rows in the history table.

Benchmark Graph

GET /benchmark-graph

Returns:

metrics/output/benchmark.png

If the graph has not been generated yet, this endpoint returns 404.

Run Prompt

POST /run

Request:

```
{
  "text": "Explain neural networks in detail.",
  "max_new_tokens": 64,
  "precision": "auto",
  "audit": true
}
```

Example:

```
curl -X POST http://127.0.0.1:8000/run \
  -H "Content-Type: application/json" \
  -d '{"text":"Explain neural networks in
detail.", "max_new_tokens":64, "precision":"auto", "audit":true}'
```

Response shape:

```
{
  "output": "generated text",
  "stats": {
    "label": "UI RUN",
    "latency": 1.23,
    "tps": 45.6,
    "disk": 0,
    "kv_hit": false,
    "kv_hit_rate": 0.0,
    "tokens_generated": 64,
    "tokens_computed": 72,
    "tokens_reused": 0,
    "compute_avoided_pct": 0.0,
    "cache_validation": false,
    "validation_latency": 0.0,
    "precision": "fp16",
    "requested_precision": "auto",
    "execution_mode": "full",
    "backend": "cpu",
    "memory_mb": 0.0
  }
}
```

Execution Record

```
GET /api/execution/latest
GET /api/execution/{request_id}
```

Returns the public-safe execution inspector record.

Audit Trace

```
GET /audit/{request_id}
GET /audit/{request_id}/export?format=json
GET /audit/{request_id}/export?format=csv
```

Returns or exports a redacted audit trace when audit mode was enabled for that request.

What The Metrics Mean

Latency

Total time for the run in seconds.

Tokens/sec

Generated tokens divided by latency.

KV Hit Rate

Ratio of cache hits to total cache lookups in the current process.

Compute Avoided

High-level percentage of prompt work served from reusable state.

Precision

The requested precision can be auto, fp16, int8, or int4. The UI history shows the actual precision used after local hardware and dependency checks.

Cache Validation

Validation mode creates local validation records without exposing prompts, outputs, token traces, or private runtime details. Enable it when starting the server:

```
CONVERA_VERIFICATION_MODE=1 uvicorn ui.app:app --reload
```

Audit Mode

Audit mode creates redacted local traces. Enable it per request with the checkbox or globally:

```
CONVERA_AUDIT_MODE=1 uvicorn ui.app:app --reload
```

KV Status

The history table shows hit when the prompt state was found in cache and miss when the runtime performed a full path.

UI Workflow For Demos

Recommended demo:

1. Start the server.
2. Open the UI.
3. Run the default prompt once.
4. Run the same prompt again.
5. Watch the KV status and hit rate.
6. Change the prompt slightly.
7. Compare latency and cache behavior.
8. Run `python -m metrics.benchmark_runner`.
9. Refresh the benchmark graph panel.

Keep public demos focused on:

- Cache hit status.
- Latency.

- Tokens/sec.
- Response.

Do not show token traces, cache internals, or private architecture.

Troubleshooting

Browser shows 404

Use:

```
http://127.0.0.1:8000/static/index.html
```

UI is visible but Run fails

Check the server terminal. Common causes:

- Model missing from models/llama3.
- Dependencies not installed.
- Model requires more memory than available.

Metrics are empty

Metrics appear after the first run. Click Run, then click Refresh.

Server reloads repeatedly

Stop the server with Ctrl-C and restart:

```
uvicorn ui.app:app --reload
```